ТЕМА 3. Вход и изход на данни в съвременния C++

Цел: Усвояване на конзолния вход и изход, форматирането на данни, работата със символи и низове и обработката на невалиден вход чрез стандартната библиотека на C++.

Какво ще упражните: `std::cin`, `std::cout`, `std::cerr`, `std::clog`, `std::setw`, `std::setfill`, `std::setprecision`, `std::fixed`, `std::scientific`, `std::getline`, `get()`, `put()`, `std::getchar`, `std::putchar` и файлови потоци.

1. Входно-изходни потоци и стандартна библиотека

В C++ входът и изходът се реализират чрез **потоци** — обекти, през които програмата чете или записва данни. Един и същ подход се използва за конзола, файлове и други източници или получатели. Операторът `>>` обикновено чете форматирани данни, а `<<` извежда данни. Методи като `get()`, `put()` и `std::getline()` дават по-точен контрол при четене на символи и цели редове.

Поток	Предназначение
`std::cin`	Стандартен вход (обикновено клавиатура).
`std::cout`	Стандартен изход (обикновено терминал).
`std::cerr`	Стандартен поток за диагностични съобщения и грешки.
`std::clog`	Поток за диагностични съобщения, който обикновено е буфериран.

Основните стандартни заглавни файлове са `<iostream>` за потоци, `<iomanip>` за манипулатори, `<string>` за низове, `<sstream>` за четене от низ, `<fstream>` за файлове и `<limits>` за граници на числови типове. Съвременният C++ използва `int main()` и имената в пространството `std` (например `std::cout`), а не остарелите `iostream.h`, `iomanip.h` и `void main()`.

Минимален пример:

```
#include <iostream>

int main() {
    int age{};
    std::cout << "Въведете възраст: ";
    if (std::cin >> age) {
        std::cout << "Възраст: " << age << '\n';
    } else {
        std::cerr << "Невалидно цяло число.\n";
        return 1;
    }
}
```

`'\n'` отпечатва нов ред; `std::endl` отпечатва нов ред **и принудително изпразва буфера**. Затова за обичайно извеждане предпочитайте `'\n'`, а `std::endl` използвайте, когато е нужно незабавно изпращане на изхода.

2. Форматни манипулатори и състояние на потока

Манипулатор / средство	Действие
`std::dec`, `std::hex`, `std::oct`	Десетична, шестнадесетична или осмична бройна система за цели числа.
`std::setw(n)`	Минимална ширина само за **следващото** извеждано поле.
`std::setfill(ch)`	Символ за запълване; остава активен, докато не бъде сменен.
`std::left`, `std::right`	Подравняване; настройката се запазва.
`std::setprecision(n)`	При `defaultfloat`: общ брой значещи цифри; при `fixed` и `scientific`: цифри след десетичната точка.
`std::fixed`, `std::scientific`	Десетичен или експоненциален формат на реални числа.
`std::defaultfloat`	Възстановява обичайния формат за реални числа.
`std::showpoint`, `std::noshowpoint`	Показва / отменя задължителната десетична точка и нужните нули.
`std::showpos`, `std::noshowpos`	Показва / отменя знак `+` пред неотрицателните числа.
`std::flush`, `std::endl`	Изпращат буферирания изход; `endl` добавя и нов ред.
`std::ends`	Добавя нулев символ `\0` към потока; рядко е нужен за конзола.

Важно: `std::setw()` се задава пред всяко поле поотделно; `std::setfill()`, системата за представяне на числата и подравняването остават в сила до следваща промяна. Форматирането на един поток не променя автоматично настройките на друг.

Задача 3.1. Ширина, подравняване, бройна система и запълване

Разгледайте програмата и обяснете как се променя представянето на числата 10 и 15.

```
#include <iomanip>
#include <iostream>

int main() {
    std::cout << 10 << 15 << "\n";
    std::cout << 10 << std::setw(10) << 15 << '\n';
    std::cout << 10 << std::hex << std::setw(10) << 15 << "\n";

    std::cout << 10 << std::dec << std::left << 15;
    std::cout << 10 << std::setw(10) << std::hex << 15 << "\n";

    std::cout << std::right << std::setfill('-');
    std::cout << 10 << std::setw(10) << 15 << '\n';
    std::cout << 10 << std::hex << std::setw(10) << 15 << "\n";
    std::cout << 10 << std::dec << std::left << 15 << "\n";
}
```

Наблюдение: Първият ред съединява `10` и `15`. `std::setw(10)` запазва общо поне 10 позиции за следващото число. При `std::hex` числото 15 става `f`. `std::left` премества запълването вдясно, а `std::setfill('-')` заменя интервалите с тирета. След задачата върнете `std::dec` и обикновен символ за запълване.

Задача 3.2. Закръгляване и точност на реални числа

Изведете `4.567` без форматиране и с 1, 2, 4 и 5 знака след десетичната точка.

```
#include <iomanip>
#include <iostream>

int main() {
    const double value = 4.567;
    std::cout << value << '\n';
    std::cout << std::fixed;
    std::cout << std::setprecision(1) << value << "\n";
    std::cout << std::setprecision(2) << value << "\n";
    std::cout << std::setprecision(4) << value << "\n";
    std::cout << std::setprecision(5) << value << "\n";

    std::cout << std::defaultfloat;
    std::cout << std::setprecision(2) << value << "\n";
}
```

Очакван изход:

```
4.567
4.6
4.57
4.5670
4.56700
4.6
```

Извод: Само `std::setprecision(2)` без `std::fixed` означава **две значещи цифри**, а не две цифри след точката. При `std::fixed` задава **цифрите след десетичната точка**. При закръгляване резултатът зависи и от двоичното представяне на стойностите с плаваща точка.

Задача 3.3. Форматиране на цена в различни режими

Изведете стойността на променливата `price = 78.5779008` в обичаен, фиксиран и експоненциален формат. Възстановете първоначалния режим накрая.

```
#include <iomanip>
#include <iostream>

int main() {
    const double price = 78.5779008;

    std::cout << "По подразбиране: " << price << "\n";
    std::cout << std::showpoint << std::setprecision(2);
    std::cout << "С 2 значещи цифри: " << price << "\n";
    std::cout << std::setprecision(5);
    std::cout << "С 5 значещи цифри: " << price << "\n";

    std::cout << std::noshowpoint << std::fixed << std::setprecision(5);
    std::cout << "Фиксиран формат: " << price << "\n";

    std::cout << std::scientific << std::setprecision(5);
    std::cout << "Експоненциален формат: " << price << "\n";
    std::cout << std::setprecision(2);
    std::cout << "Експоненциален формат (2): " << price << "\n";

    std::cout << std::defaultfloat << std::setprecision(6);
    std::cout << "Възстановен формат: " << price << "\n";
}
```

Примерен изход:

```
По подразбиране: 78.5779
С 2 значещи цифри: 79.
С 5 значещи цифри: 78.578
Фиксиран формат: 78.57790
Експоненциален формат: 7.85779e+01
Експоненциален формат (2): 7.86e+01
Възстановен формат: 78.5779
```

`std::cerr` използвайте за **съобщения за грешки**, например `std::cerr << "Неуспешно четене.\n";`. Самото извеждане в `std::cerr` не означава, че е възникнала грешка. За връщане към обичайния формат използвайте `std::defaultfloat` и подходяща точност (обикновено 6 значещи цифри), а не `precision(0)`.

Задача 3.4. Таблица с резултати на футболни отбори

Изведете пет отбора и точките им за три седмици. Използвайте минимална ширина на колоните и цикли вместо повтаряне на еднакви оператори.

```
#include <iomanip>
#include <iostream>
#include <string>

int main() {

    std::cout << std::setw(10) << "Левски" << std::setw(10) << "ЦСКА" << std::setw(10)
      << "Славия" << std::setw(10) << "Спартак" << std::setw(10) << "Ботев" << "\n";

    std::cout << std::setw(10) << 2 << std::setw(10) << 5 << std::setw(10)
      << 3 << std::setw(10) << 0 << std::setw(10) << 4 << "\n";
    std::cout << std::setw(10) << 6 << std::setw(10) << 3 << std::setw(10)
      << 1 << std::setw(10) << 0 << std::setw(10) << 2 << "\n";
    std::cout << std::setw(10) << 1 << std::setw(10) << 1 << std::setw(10)
      << 2 << std::setw(10) << 2 << std::setw(10) << 1 << "\n";
}
```

Извод: `std::setw(10)` се поставя **преди всяка клетка**. Имената в примера са транслитерирани, за да се подравняват предвидимо в обикновен терминал. При текст на кирилица в UTF-8 ширината в байтове може да се различава от броя видими букви; за сложни Unicode таблици е нужна допълнителна обработка.

3. Въвеждане на числа и проверка на входа

Операторът `std::cin >> value` прочита стойност от очаквания тип и пропуска началните празни символи (интервал, табулация и нов ред). Ако срещне невалиден символ, потокът влиза в състояние на неуспех. **Никога не използвайте неинициализирана променлива след провалено четене.**

Задача 3.5. Числа с интервали и запетаи

Проверете какво става при вход `5,6,7` и сравнете с вход `5 6 7`.

```
#include <iostream>

int main() {
    int a, b, c;
    std::cout << "Въведете три цели числа: ";

    std::cin >> a >> b >> c;
    std::cout << "Получените стойности са: " << a << ' ' << b << ' ' << c << '\n';
}
```

При `5,6,7` първото четене получава `5`, след което следващото не може да преобразува запетаята в цяло число. Въвеждането с `5 6 7` е успешно. Входът от клавиатурата е буфериран. Всяка последователност от натиснати клавиши се разглежда като пакет, който се обработва след като се натисне клавишът Enter. Няма значение дали в примера трите числа ще се въведат на един ред разделени с интервал или всяко число написано на отделен ред.

При смесените типове `int a; float b; int c;` и вход `10.5 23.1 15` последователното форматирано четене може да получи `a = 10`, `b = 0.5`, `c = 23`: всяко извличане чете само символите, валидни за съответния тип. Това не е удобен формат за надеждно въвеждане, затова е по-добре входът да се проверява като цял ред.

4. Низове, символи и буфериран вход

`std::cin >> text` прочита една дума и спира при празен символ. `std::getline(std::cin, text)` прочита **целия ред**, включително интервалите, но без крайния символ `\n`. За съвременни програми предпочитайте `std::string` вместо фиксирани масиви от `char`.

Задача 3.6. Въвеждане на град

Въведете `Варна10`, а след това `Черноморска столица`. Проверете какво се запазва в низа.

```
#include <iostream>
#include <string>

int main() {
    std::string city;
    std::cout << "Въведете град: ";

    if (std::getline(std::cin, city)) {
        std::cout << "Въведохте: " << city << '\n';
    } else {
        std::cerr << "Неуспешно въвеждане.\n";
        return 1;
    }
}
```

Извод: `std::string` няма ръчно зададен лимит от 6 или 25 байта в този пример. При стария вариант `char city[6]` има място за най-много **5 байта данни и `\0`**. В UTF-8 буквите на кирилица заемат повече от един байт, така че не трябва да приравнявате размера на масива с броя видими букви. `cin.getline(buffer, capacity)` може да зададе `failbit`, ако редът е твърде дълъг.

Задача 3.7. Четене на отделни символи с `get()`

Обяснете стойностите на `c1`, `c2` и `c3` при въвеждане на два реда: `AB` и `CD`.

```
#include <iostream>

int main() {
    char c1, c2, c3;
    std::cin.get(c1);
    std::cin.get(c2);
    std::cin.get(c3);

    std::cout << "c1 = " << c1 << '\n';
    std::cout << "c2 = " << c2 << '\n';
    if (c3 == '\n') {
        std::cout << "c3 = символ за нов ред (\\n)\n";
    } else {
        std::cout << "c3 = " << c3 << '\n';
    }
}
```

Отговор: `c1` е `A`, `c2` е `B`, а `c3` е `\n` от първото натискане на Enter.

Функцията get() е за буферирано въвеждане. Когато въвеждате символи, те не се предават незабавно към вашата програма, а се записват в буфер. Буферът се пълни със символи, докато се натисне Enter. Затова ако зададете въпрос:

„Желаете ли да видите резултата отново (Y/N)?“ и използвате функцията get() за въвеждане, програмата не приема натискането на Y, докато не се натисне Enter.

Тази е причината променливата c3 да получи стойност '\n'.

С използване на буфериран вход потребителят може да въведе низ от символи за обработка с get() в цикъл и да поправя входните данни с натискане на клавиша Backspace преди Enter.

Задача 3.8. Повторение на входа като „ехо“

Прочетете един ред **символ по символ** и го изведете обратно, докато се достигне Enter.

```
#include <iostream>

int main() {
    char symbol;
    std::cout << "Въведете един ред символи:\n";
    do {
      std::cin.get(symbol);
      std::cout.put(symbol);
    } while (symbol != '\n') {

    std::cout << "\nТова беше демонстрацията!\n";
}
```

При вход `Обучение по C++` програмата повтаря текста с интервалите му. `std::cout.put(symbol)` извежда **един символ**. В терминала самото въвеждане обикновено се показва и от терминала; това не е допълнителният изход от програмата. Правете разлика между символа '\n' и стринга "\n". Действието е едно и също, но типовете на данните са различни: char и символен низ (константа). Във варианта на програмата е използван оператор за цикъл do...while

Задача 3.9. Четни по позиция символи

Какво ще се изведе от следната програма при вход един ред във вида: 0 1 2 3 4 5 6 7 8 9 10 11?

```
#include <cctype>
#include <iostream>
#include <string>

int main() {
    std::string line;
    std::getline(std::cin, line);

    for (int i = 0; i < line.size(); i += 2) {
        std::cout << line[i];
    }
    std::cout << '\n';

    int count = 0;
    for (int i = 0; i < line.size(); i++) {
      if (line[i] != ' ') {
        if (count % 2 == 0)
            std::cout << line[i];
        count++;
      }
    }
    std::cout << '\n';
}
```

Резултати:

```
01234567891 1
0246811
```

Програмата извежда само четни по ред символи и така празните интервали между числата се изключват. Двете последни числа 10 и 11 се състоят от 2 цифри и затова 0 на 10 се игнорира (явява се четен символ), извежда се празния интервал между тях и втората единица на числото 11 (тя също е четен символ).

Задача 3.10. Повторно въвеждане до потвърждение

Програмата да приема цяло число и да пита дали е вярно. При отговор `no` (или `n`) да изисква ново число, а при `yes` (или `y`) да го приема. Невалидното число не бива да се приема.

```
#include <iostream>
#include <sstream>
#include <string>

bool getNumber(int& number) {
    std::string line;

    while (true) {
        std::cout << "Въведете цяло число: ";
        std::getline(std::cin, line);

        std::istringstream input(line);
        int candidate;
        std::string extra;
        if (!(input >> candidate) || (input >> extra)) {
            std::cout << "Невалидно цяло число. Опитайте отново.\n";
            continue;
        }

        std::cout << "Въведохте " << candidate
                  << ". Вярно ли е? (yes/no): ";
        std::string answer;
        std::getline(std::cin, answer);

        if (answer == "yes" || answer == "y") {
            number = candidate;
            return true;
        }
        if (answer != "no" && answer != "n") {
            std::cout << "Отговорете с yes или no.\n";
        }
    }
}

int main() {
    int number;
    getNumber(number);
    std::cout << "Крайната стойност е " << number << '\n';
}
```

Пример: първо се въвежда `57`, следва отговор `no`, после се въвежда `75` и отговор `yes`. Резултатът е `Крайната стойност е 75`.

Извод: Тук `std::getline()` приема един ред, а `std::istringstream` проверява дали той съдържа **точно едно** цяло число. Не е нужна отделна функция за изчистване на оставащите символи. При смесване на `std::cin >>` и `std::getline()` може да се използва `std::cin.ignore(10000, '\n')`, след като състоянието на потока е валидно. Тази функция ще „работи“ за редове с до 10000 символа (числото може да се промени) и ще игнорира символа за нов ред.

5. Вход/изход към файлове и C-функции за символи

В старите среди `getch()`, `getche()`, `putch()` и заглавният файл `<conio.h>` са били удобни за четене на клавиши без Enter. Те **не са част от стандартния C++** и не са преносими. Съвременният вариант на задачата използва стандартен, буфериран вход. За скриване на натиснатите клавиши са необходими специфични за операционната система интерфейси.

Задача 3.11. Десет символа: въвеждане, екран и файл

Въведете точно 10 латински букви на един ред, повторете ги на екрана и ги запишете във файла `bukvi.txt`. Това запазва основната логика на оригиналната задача, като заменя стария принтерен поток `PRN` със стандартен файлов изход.

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::string letters;
    std::cout << "Въведете 10 латински букви: ";
    if (!std::getline(std::cin, letters) || letters.size() != 10) {
        std::cerr << "Трябва да въведете точно 10 ASCII символа.\n";
        return 1;
    }

    std::cout << "На екрана: " << letters << '\n';

    std::ofstream output("bukvi.txt");
    if (!output) {
        std::cerr << "Файлът не може да се отвори.\n";
        return 1;
    }
    for (char symbol : letters) {
        output.put(symbol);
    }
    if (!output) {
        std::cerr << "Грешка при запис във файла.\n";
        return 1;
    }
    std::cout << "Записът приключи успешно.\n";
}
```

Пример: вход `abcdefghij` → същият текст на екрана и във файла `bukvi.txt`. `std::ofstream` е стандартен изходен файлов поток. Проверката `letters.size() == 10` измерва **байтове**; тя е подходяща за зададените латински букви, но не е общ Unicode брояч на видими символи.

Задача 3.12. По една точка за всеки въведен символ

Напишете програма, която чете символите от реда и извежда точка за всеки прочетен символ, включително крайния `\n`.

```
#include <cstdio>

int main() {
    std::puts("Въведете последователност от символи:");

    int symbol;
    while ((symbol = std::getchar()) != EOF) {
        std::putchar('.');
        if (symbol == '\n') {
            break;
        }
    }
    std::putchar('\n');
}
```

Резултат: При вход `abcde` и Enter се извеждат **6 точки** — пет за буквите и една за `\n`. Използвайте `int`, а не `char`, за стойността, върната от `std::getchar()`, защото `EOF` трябва да се различава от валиден байт. Функциите са декларирани в `<cstdio>`.

6. Въпроси и задачи за самостоятелна работа

1. Каква е грешката в `char symbol[80]; std::putchar(symbol);`? Покажете как се извежда отделен символ (`symbol[0]`) и как се извежда коректно завършен низ (`std::cout << symbol`).

2. Сравнете `std::getchar()`, `std::cin.get()` и нестандартната `getch()`. Кои от тях са преносими и кои могат да четат символа за нов ред?

3. Как се различават `std::cin.get(ch)` и `std::cin >> ch` при вход, съдържащ интервали?

4. Напишете програма, която чете пълно име с интервали чрез `std::getline()`.

5. Какво извежда `std::puts("Hello")`? Кога е по-удобно да използвате `std::cout << "Hello\n";`? Може ли директно да подадете един `char` на `std::puts()`?

6. Коя комбинация от манипулатори задава точно три цифри след десетичната точка?

7. Кой заглавен файл е необходим за `std::setw`, `std::setfill` и `std::setprecision`?

8. Какво е предимството на `std::getline(std::cin, s)` пред `std::cin >> s`, когато входът съдържа интервали?

9. Намерете и поправете грешката в следния фрагмент. Изяснете дали между въведения низ и надписа „Край на програмата“ има нов ред.

```
#include <iostream>
#include <string>

int main() {
    std::string s;
    std::cout << "Въведете низ: " << std::endl <;
    std::getline(std::cin, s);
    std::cout << s << "Край на програмата";
}
```

10. Напишете програма, която прочита ASCII символ и извежда кода му в десетична и шестнадесетична бройна система (`std::dec`, `std::hex`). За да не получите отрицателна стойност при преобразуване на `char`, използвайте `static_cast<unsigned char>(ch)` преди преобразуването до `int`.

11. Създайте „машинописна машинка“: прочитайте символите от един ред със `std::cin.get()` и ги извеждайте със `std::cout.put()` до Enter или край на входа.

12. Въведете 10 латински букви и ги изведете в обратен ред. Използвайте `std::string` и цикъл или `std::reverse()` от `<algorithm>`.

13. Проверете дали дадена дума е палиндром, като сравнявате символите от двата края към средата. За началния вариант използвайте латински букви; за UTF-8 текст са необходими допълнителни правила за Unicode.

14. Покажете положителните числа със знак `+`, като използвате `std::showpos`; после върнете обичайния режим с `std::noshowpos`.

15. Изведете следните пет реда чрез форматиране със `std::setw`, `std::setfill`, `std::left`, `std::right`, `std::fixed` и `std::setprecision`:

```
Hello
%%%%%Hello
Hello%%%%%
123.234567
123.235%%%
```

16. Изведете таблица за числата от 2 до 100 с **натурален логаритъм** `std::log(x)` и **десетичен логаритъм** `std::log10(x)` от `<cmath>`. Използвайте `std::fixed`, `std::setprecision(5)` и `std::setw(10)` за подравнени стойности.

7. Обобщение

- За конзолен вход/изход използвайте `<iostream>` и обектите от `std`.
- За форматиран изход използвайте `<iomanip>`; разграничете значещи цифри от цифри след десетичната точка.
- За текст с интервали предпочитайте `std::string` и `std::getline()`.
- Проверявайте всяко потенциално неуспешно четене и не използвайте неинициализирани стойности.
- При символни операции различавайте `>>`, `get()`, `put()` и `getchar()`/`putchar()`.
- За файлове използвайте `<fstream>`; избягвайте непереносими конструкции като `<conio.h>`, `getch()` и устройството `PRN`.